

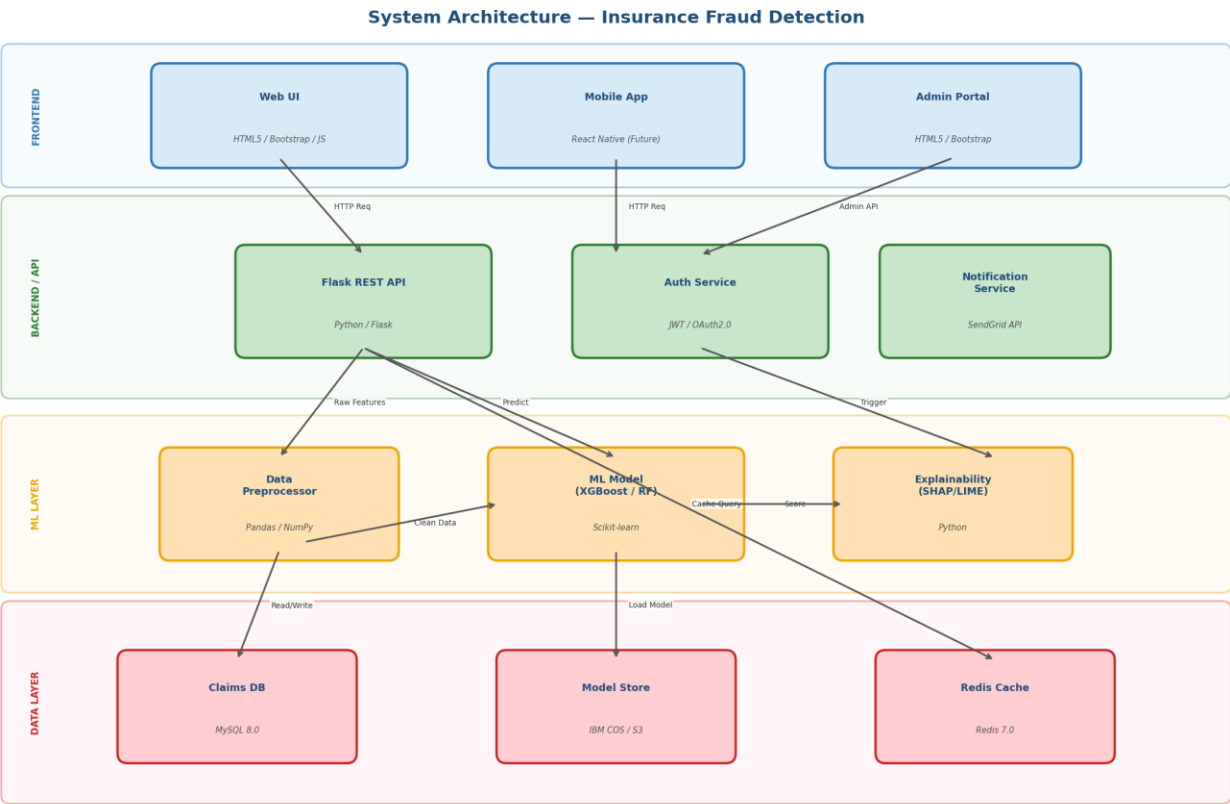
Technology Stack

Architecture & Stack — Design Phase II

Date	07 March 2026
Team ID	
Project Name	Insurance Fraud Detection Using Machine Learning
Maximum Marks	4 Marks

Technical Architecture

The architectural diagram below shows all layers of the Insurance Fraud Detection system — from the frontend user interface through to the ML prediction engine and data storage. The system follows a 3-tier architecture with clear separation between the presentation, application logic, and data layers.



Architecture Overview

Layer	Description	Technology
Frontend Layer	User-facing web interface for claim submission, fraud score display, and analytics dashboard. Mobile-responsive.	HTML5, CSS3, Bootstrap 5, JavaScript, Chart.js
API / Backend Layer	RESTful API server handling business logic, authentication, routing, and ML model communication.	Python 3.10, Flask, Flask-RESTful, JWT
ML Prediction Layer	Data preprocessing pipeline and ML models for fraud classification and explainability.	Scikit-learn, XGBoost, Pandas, NumPy, SHAP

Layer	Description	Technology
Data Storage Layer	Relational database for users and claims; object storage for ML model artefacts; cache for performance.	MySQL 8.0, IBM Cloud Object Storage, Redis
Notification Layer	External email service for fraud alerts and daily summaries.	SendGrid API / SMTP
Infrastructure Layer	Cloud deployment with load balancing and container orchestration.	IBM Cloud / Docker / Nginx / Kubernetes (future)

Table 1 — Components & Technologies

S. No	Component	Description	Technology
1	User Interface (Web)	Primary web-based interface where claimants submit claims and officers review fraud scores and analytics.	HTML5, CSS3, Bootstrap 5, JavaScript (Vanilla / jQuery)
2	Admin Portal	Separate interface for administrators to manage users, monitor model performance, and configure thresholds.	HTML5, Bootstrap 5, Chart.js for metrics visualisation
3	REST API — Application Logic	Core Flask application handling HTTP request routing, input validation, authentication middleware, and response formatting.	Python 3.10, Flask 2.x, Flask-RESTful, Flask-JWT-Extended
4	Data Preprocessing Pipeline	Automated feature engineering pipeline: handles missing values, categorical encoding (One-Hot/Label), feature scaling, outlier removal, and normalisation.	Python, Pandas, NumPy, Scikit-learn Pipelines (ColumnTransformer)
5	ML Fraud Classification Model	Supervised classification model trained on labelled historical claim data. Outputs fraud probability score and risk label.	Scikit-learn (Random Forest, Logistic Regression), XGBoost, Joblib (model serialisation)
6	Explainability Module (XAI)	Generates per-prediction feature importance scores to explain why a claim was classified as fraud or genuine.	SHAP (SHapley Additive exPlanations), Python
7	Primary Database	Relational database storing users, claims, prediction logs, audit records, and model version metadata.	MySQL 8.0 — with indexed tables for claims and predictions
8	Cloud Database Service	Managed cloud database for high availability with automatic backups and failover.	IBM DB2 on Cloud / AWS RDS (MySQL)
9	Model Artefact Storage	Object storage for serialised ML model files (.pkl), scaler objects, and	IBM Cloud Object Storage / AWS S3

S. No	Component	Description	Technology
		encoder mappings. Versioned by model ID.	
10	Cache Layer	In-memory cache for frequently accessed claim queries and pre-computed analytics dashboard data to reduce DB load.	Redis 7.0
11	External API — Email Alerts	Transactional email service for sending fraud alert notifications to claims officers.	SendGrid Email API (RESTful)
12	External API — Authentication	OAuth 2.0 provider for user sign-in via Google account.	Google OAuth 2.0
13	Infrastructure — Web Server	Production-grade WSGI server for running the Flask application. Nginx as reverse proxy.	Gunicorn / uWSGI + Nginx
14	Infrastructure — Cloud Deployment	Container-based cloud deployment with auto-scaling capabilities.	Docker, IBM Cloud Foundry / AWS EC2, Kubernetes (future)

Table 2 — Application Characteristics

S. No	Characteristic	Description	Technology Used
1	Open-Source Frameworks	All core development frameworks are open-source. Python, Flask, Scikit-learn, XGBoost, Pandas, Bootstrap, and Redis are all fully open-source with active communities.	Python 3.10, Flask 2.x, Scikit-learn 1.x, XGBoost 2.x, Pandas, NumPy, Bootstrap 5, Redis 7.0
2	Security Implementations	HTTPS/TLS 1.3 encryption for all data in transit. bcrypt for password hashing (never plain-text). JWT tokens for stateless session management with 24-hour expiry. RBAC enforced on all API routes via Flask decorators. SQL injection prevention via ORM (SQLAlchemy). OWASP Top 10 addressed in code reviews. Rate limiting on authentication endpoints.	HTTPS/TLS 1.3, bcrypt, JWT (flask-jwt-extended), SQLAlchemy ORM, Flask-Limiter, OWASP Guidelines, Role-Based Access Control (RBAC)
3	Scalable Architecture	3-tier architecture (Presentation / Application / Data) with fully independent layers. Flask Blueprints used for modular, independently deployable service units. Stateless API design allows horizontal scaling of the API layer without shared state. ML prediction service can be extracted as a separate micro-service in future.	3-Tier Architecture, Flask Blueprints, Stateless REST API, Docker (containerisation), Kubernetes (future horizontal scaling)
4	Availability	Application deployed on cloud with a minimum of 2 active instances behind an Nginx load balancer for traffic distribution and failover. Cloud auto-scaling triggered at >70% CPU utilisation. Recovery Time Objective (RTO): <1 hour. Recovery Point Objective (RPO): <24 hours via automated daily backups.	Nginx Load Balancer, Cloud Auto-Scaling (IBM Cloud / AWS), Multi-AZ Database Deployment, Automated Backups (S3/IBM COS)
5	Performance	ML inference API designed to respond in <2 seconds for single claim predictions. Redis caching used for analytics dashboard queries and repeated claim lookups (cache TTL: 5 minutes). CDN used for all static frontend assets (JS, CSS, images). Async task queue (Celery) for batch claim processing and email dispatch to prevent blocking the main request thread.	Redis Cache, Celery Task Queue, CDN (Cloudflare), Gunicorn Multi-Worker, Database Query Indexing, Async Background Jobs